

串口通讯

串口通讯

一、学习目标

串口通讯

串口工作模式

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. 串口时钟配置

3. 串口参数配置

4. 写入程序

5. 编译

四、程序分析

五、实验现象

一、学习目标

1.学习MSPM0G3507主板串口的基本使用。

2.了解串口通讯基本知识。

串口通讯

串口通信(Serial Communication)，是指外设和计算机间通过数据信号线、地线等按位进行传输数据的一种通信方式，属于串行通信方式。

串口通信的关键参数包括波特率、数据位、停止位和奇偶校验位。为了确保通信正常，这些参数必须在通信双方保持一致。

串口工作模式

1. 单工模式 (Simplex)

- 特点: 数据只能单向传输，发送端负责发送，接收端负责接收，没有反向通信。
- 应用: 适用于只需单向传递信息的场景，如传感器的数据输出。

2. 全双工模式 (Full-Duplex)

- 特点: 数据可以同时两个方向上传输，发送和接收互不干扰。
- 实现: 需要两条数据线，分别用于发送 (TX) 和接收 (RX)。
- 应用: 常用于需要高效、实时通信的场景，如计算机与外设的交互。

3. 半双工模式 (Half-Duplex)

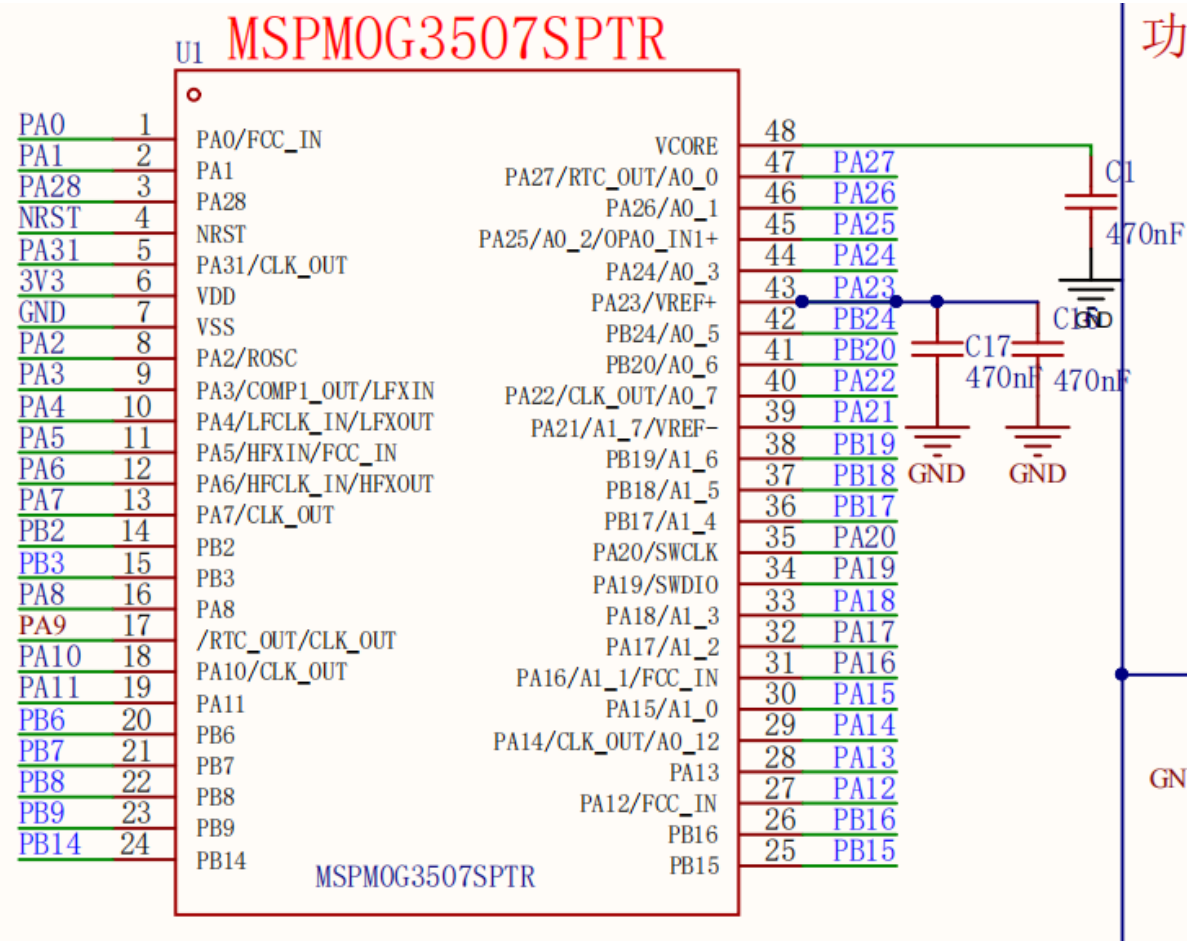
- 特点: 数据在两个方向上传输，但不能同时进行，必须交替发送和接收。
- 实现: 发送和接收共享一条通信线，依靠时序切换方向。
- 应用: 适用于对实时性要求不高的场景，如对讲机通信。

这三种模式适应了不同通信需求，可根据具体应用场景灵活选择。

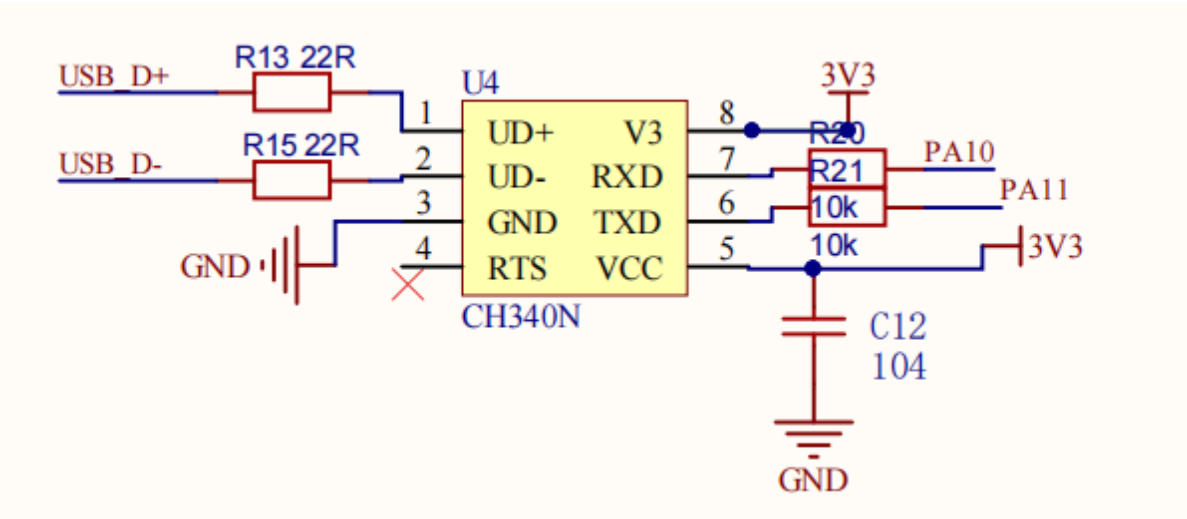
二、硬件搭建

由于开发板上自带CH340串口电路，可直接通过USB线实现串口通信，无需外接USB转TTL模块。

MSPM0G3507主控图



Type-C部分原理图



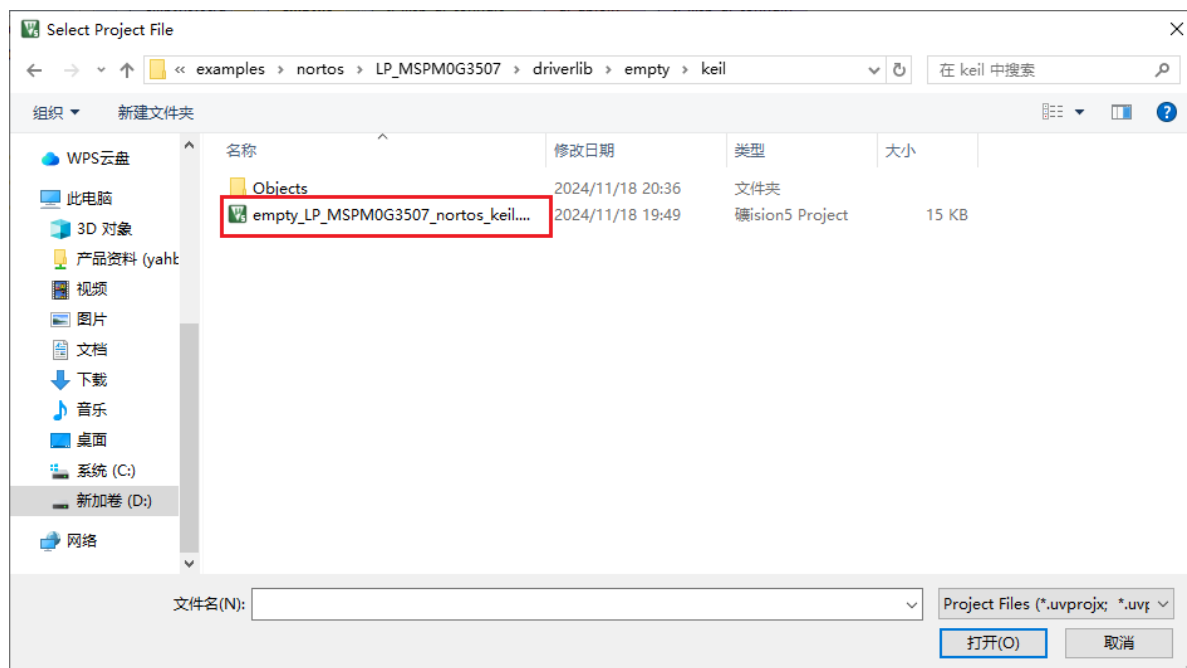
21	PA10	UART0_TX [2] / SPI0_POC1 [3] / I2C0_SDA [4] / TIMA1_C0 [5] / TIMG12_C0 [6] / TIMA0_C2 [7] / I2C1_SDA [8] / CLK_OUT [9]	56	18	14	15	高驱动
22	PA11	UART0_RX [2] / SPI0_SCK [3] / I2C0_SCL [4] / TIMA1_C1 [5] / COMP0_OUT [6] / TIMA0_C2N [7] / I2C1_SCL [8]	57	19	15	16	高驱动

三、实验步骤

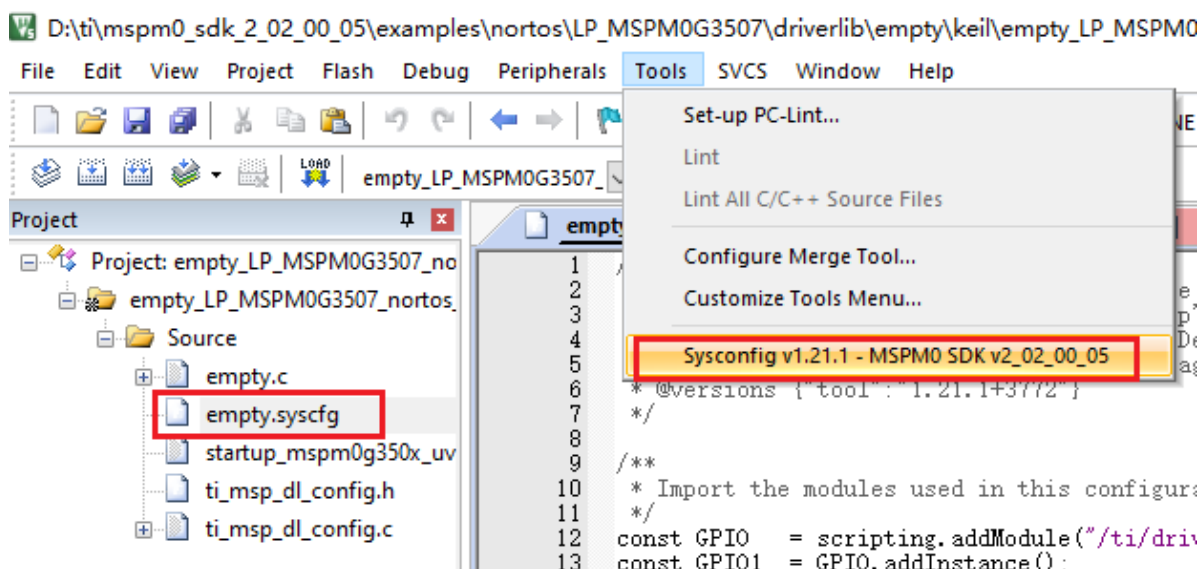
本节课使用PA10和PA11引脚上的UART0外设，搭配开发板板载的CH340这个USB转串口芯片，实现接收上位机发送过来的串口数据，再通过串口发送功能将数据发送给上位机的操作。

1. 打开SYSCONFIG配置工具

在KEIL中打开SDK中的空白工程 empty。



选中之后打开，在keil界面打开 empty.syscfg 文件，在 **empty.syscfg** 文件打开情况下，再在 Tools 栏选择打开 SYSCONFIG 的GUI界面。

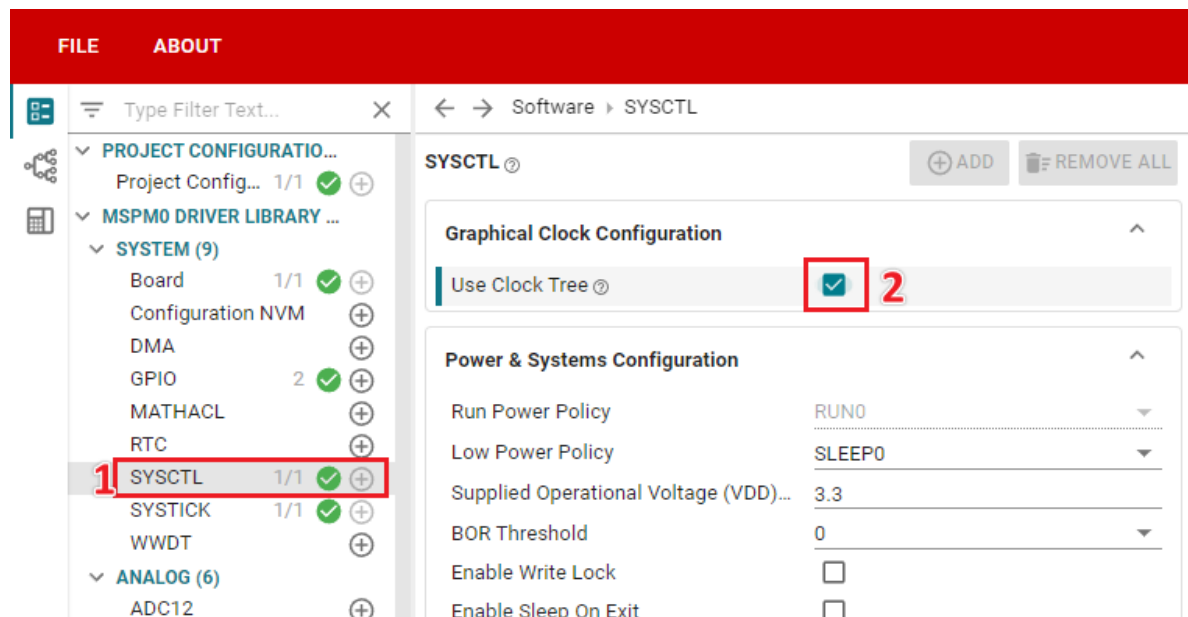


2. 串口时钟配置

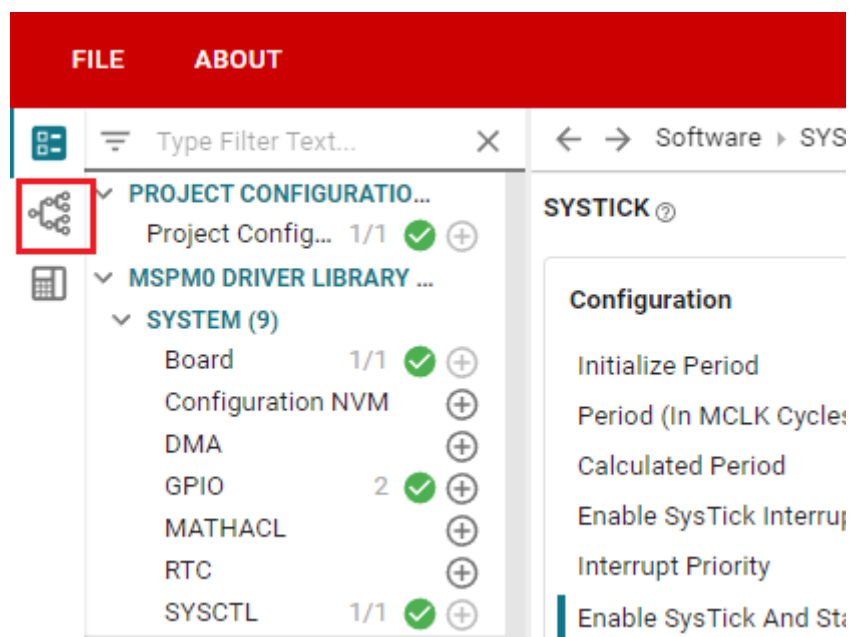
串口的时钟来源有三个：

- BUSCLK**: 由内部高频振荡器提供的CPU时钟，通常芯片出厂时设置为了32MHz。
- MFCLK**: 只能使用固定的4MHz时钟。开启的话需要配置时钟树的SYSOSC_4M分支，才能够正常开启。
- LFCLK**: 由内部的低频振荡器提供时钟（32KHz）。在运行、睡眠、停止和待机模式下有效，使用该时钟可以实现更低的功耗。

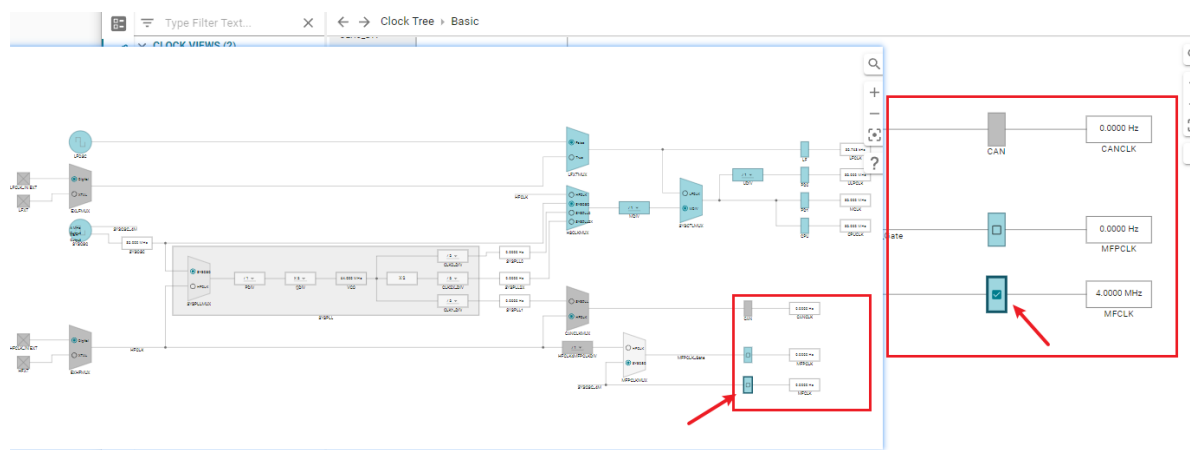
本案例使用的是MFCLK作为串口的时钟来源，开启MFCLK需要在 SYSCONFIG 中配置时钟树。在 SYSCONFIG左侧的选项卡中找到SYSTCTL选项，在选项页中找到 Use Clock Tree 进行打勾，以开启时钟树的配置。



点击 SYSCONFIG 界面最左边第二个图标，可以打开时钟树。

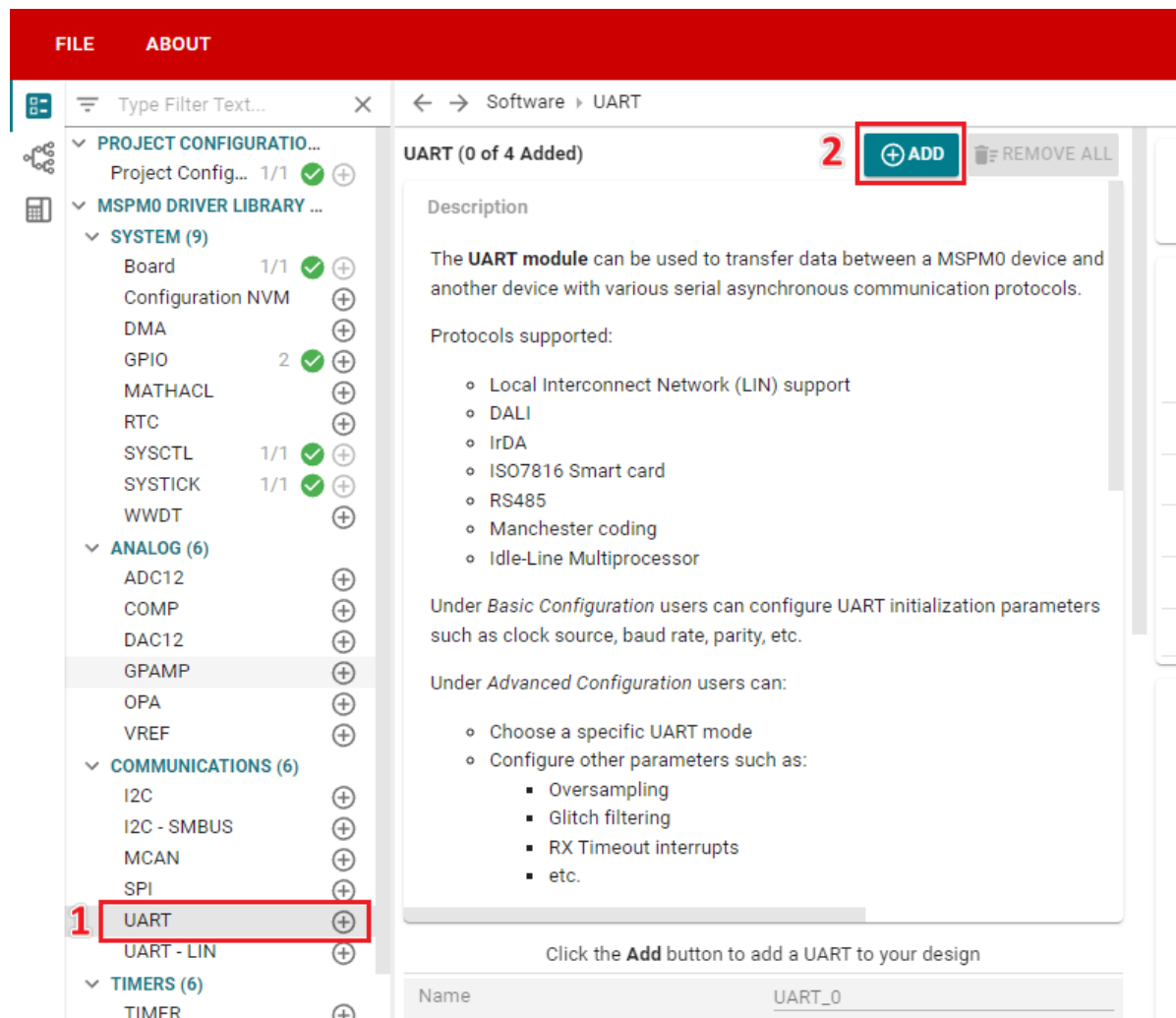


在时钟树中打开MFCLK的开关，如下图所示位置，点击SYSOSC_4M分支上的选项块，开启MFCLK时钟。当右侧出现4.0000Mhz时说明设置完成。



3. 串口参数配置

在 SYSCONFIG 中，左侧选择 MCU 外设，找到 UART 选项并点击进入。在 UART 中点击 ADD，添加一组串口外设。



基本配置：自定义串口名称，这里设置为 UART_0，Clock Source 时钟来源设置为 MFCLK，Clock Divider 时钟分频选择 Divide by 1，软件会自动计算时钟源频率。设置波特率为9600、数据位8位、停止位1位、校验位无、不使用硬件流控制的配置。

UART (1 of 4 Added) ⓘ

+

ADD

🗑️

REMOVE ALL

✓

UART_0

📄

🗑️

Name

UART_0

Selected Peripheral

UART0

Quick Profiles

^

UART Profiles

Custom

▼

Basic Configuration

^

UART Initialization Configuration

^

Clock Source

MFCLK

▼

Clock Divider

Divide by 1

▼

Calculated Clock Source

4.00 MHz

Target Baud Rate

9600

Calculated Baud Rate

9598.08

▼

Calculated Error (%)

0.02

Word Length

8 bits

▼

Parity

None

▼

Stop Bits

One

▼

HW Flow Control

Disable HW flow control

▼

高级配置：使用默认选项。采样周期 Oversampling 的选择有3、8、16等三个档，通常选择3和8即可，如果时钟偏差较大或者时钟速度太快导致波特率分频的系数计算异常，就选择16档。只要不要让 **sysconfig** 报错即可。这里我们的串口时钟频率是4MHz，所以我们选择16档。

Advanced Configuration

^

UART Mode

Normal UART Mode

▼

Communication Direction

TX and RX

▼

Oversampling

16x

▼

Enable FIFOs

☐

Analog Glitch Filter

Disabled

▼

Digital Glitch Filter

0

Calculated Digital Glitch Filter

0.00 s

RX Timeout Interrupt Counts

0

Calculated RX Timeout Interrupt

0.00 s

Enable Internal Loopback

☐

Enable Majority Voting

☐

Enable MSB First

☐

中断配置：案例使用中断方式的串口接收，每当接收到一个字节将会进入到接收中断中。

Interrupt Configuration ⓘ

^

Enable Interrupts

Receive

▼

Interrupt Priority

Default

▼

引脚配置：根据原理图可知，PA10 是 TX，PA11 是 RX，也就是串口0

PinMux Peripheral and Pin Configuration

^

UART Peripheral

Any(UART0)

▼

RX Pin

PA11/57

▼

🔒

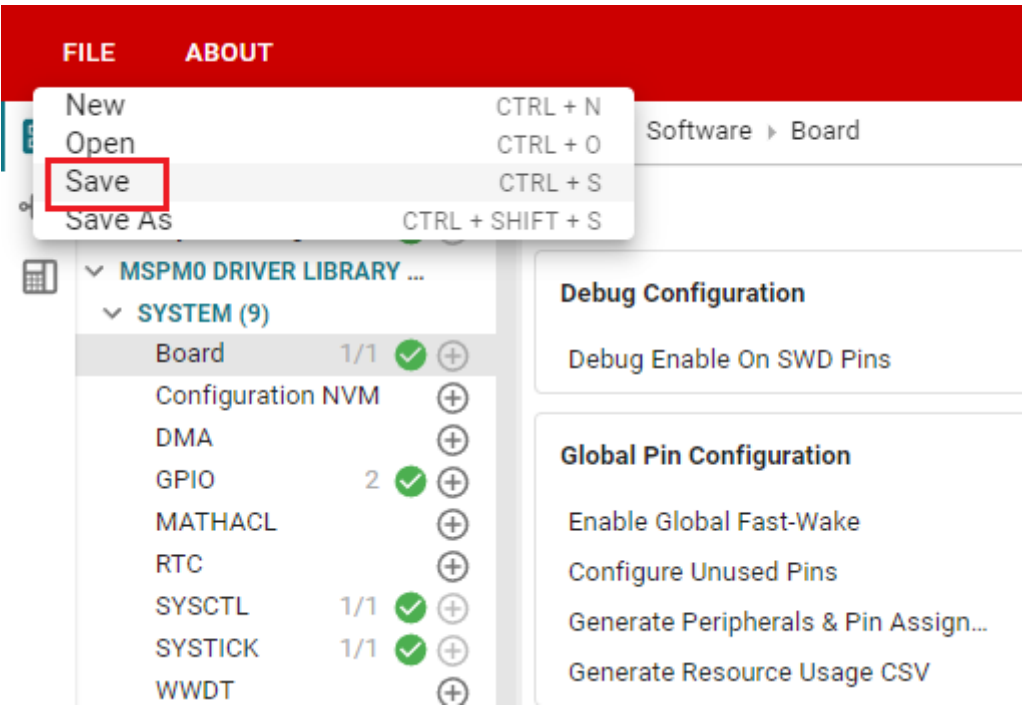
TX Pin

PA10/56

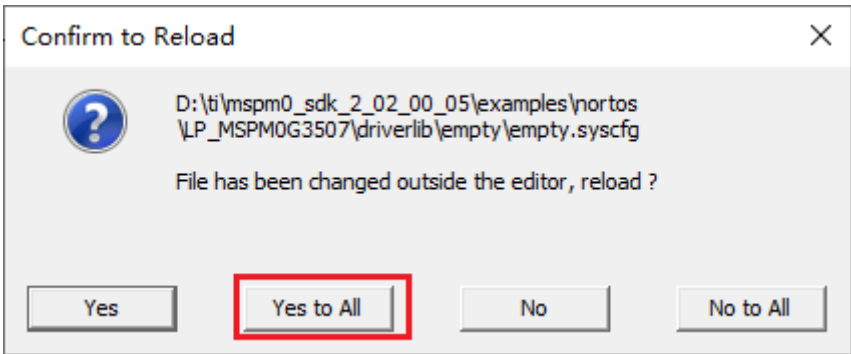
▼

🔒

点击 SAVE 将 SYSCONFIG 中的配置保存，之后关掉 SYSCONFIG 回到keil。



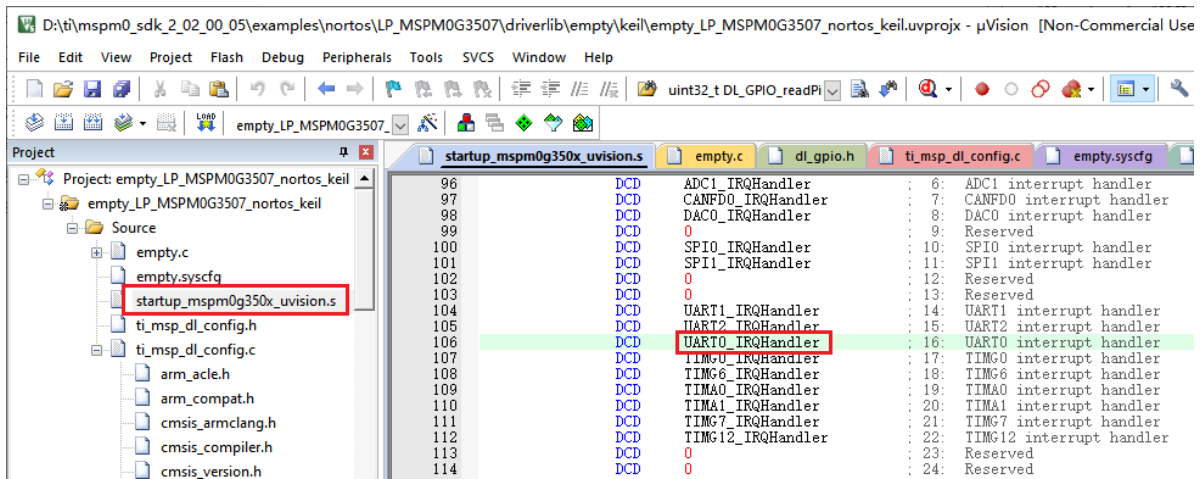
在弹窗中选择 Yes to All



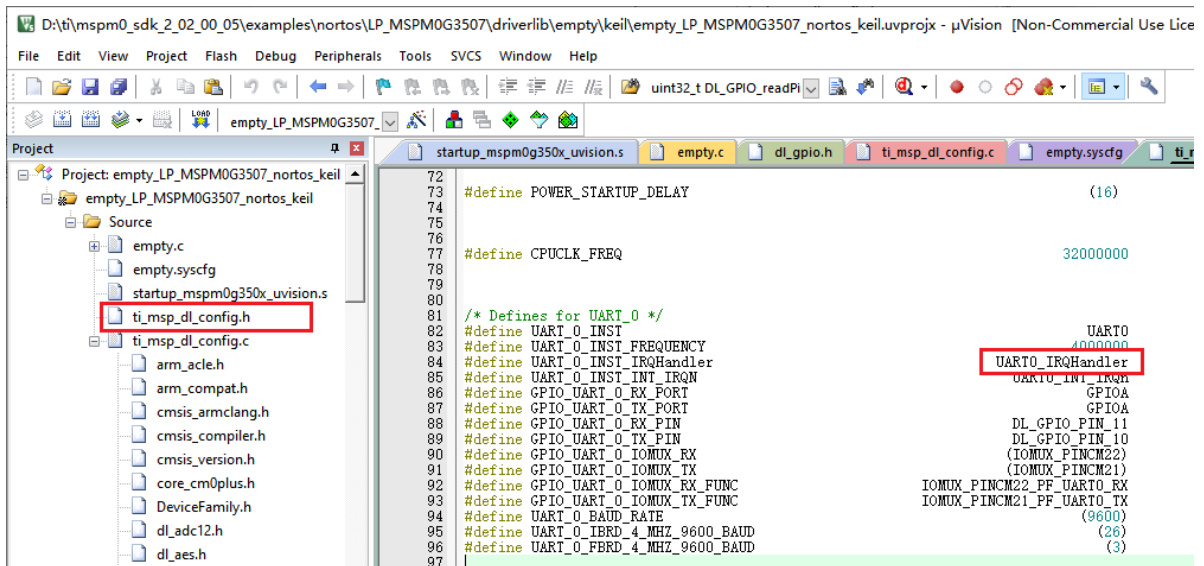
同样我们还需要确认ti_msp_dl_config.c和ti_msp_dl_config.h文件是否更新。直接编译，编译就会自动更新到keil中。如果没有更新，我们也可以把 SYSCONFIG 中的文件内容复制过来。

4. 写入程序

配置好串口之后，我们还要手动编写串口的中断服务函数。因为我们开启了串口的接收中断，当串口接收到数据时，就会触发一次中断，触发中断就会执行中断服务函数。每个中断对应的中断服务函数名称通常是固定的，不能随意修改，否则将无法正确进入中断服务函数。中断服务函数的具体名称可以在工程的 `startup_mspm0g350x_uvision.s` 文件中找到，该文件中列出了各个中断源对应的中断服务函数名称。



也可以在我们生成的ti_msp_dl_config.h文件中看到关于串口0中断服务函数的宏定义。



所以在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

volatile unsigned int delay_times = 0;
volatile unsigned char uart_data = 0;

void delay_ms(unsigned int ms);
void uart0_send_char(char ch);
void uart0_send_string(char* str);

int main(void)
{
    SYSCFG_DL_init();
    //清除串口中断标志 clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //使能串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);

    while (1)
    {
        //LED引脚输出高电平 LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(500);
        //LED引脚输出低电平 LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
    }
}
```



```

        delay_ms(500);
    }
}

//搭配滴答定时器实现的精确ms延时 Accurate ms delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

//串口发送单个字符 Send a single character through the serial port
void uart0_send_char(char ch)
{
    //当串口0忙的时候等待，不忙的时候再发送传进来的字符
    // Wait when serial port 0 is busy, and send the incoming characters when it
    is not busy
    while( DL_UART_isBusy(UART_0_INST) == true );
    //发送单个字符 Send a single character
    DL_UART_Main_transmitData(UART_0_INST, ch);
}

//串口发送字符串 Send string via serial port
void uart0_send_string(char* str)
{
    //当前字符串地址不在结尾 并且 字符串首地址不为空
    // The current string address is not at the end and the string first address
    is not empty
    while(*str!=0&&str!=0)
    {
        //发送字符串首地址中的字符，并且在发送完成之后首地址自增
        // Send the characters in the first address of the string, and increment
        the first address after sending.
        uart0_send_char(*str++);
    }
}

//滴答定时器的中断服务函数 Tick ??timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

//串口的中断服务函数 Serial port interrupt service function
void UART_0_INST_IRQHandler(void)
{
    //如果产生了串口中断 If a serial port interrupt occurs
    switch( DL_UART_getPendingInterrupt(UART_0_INST) )
    {
        case DL_UART_IIDX_RX://如果是接收中断 If it is a receive interrupt
            //接发送过来的数据保存在变量中 The data sent is saved in the variable
            uart_data = DL_UART_Main_receiveData(UART_0_INST);
            //将保存的数据再发送出去 Send the saved data again
            uart0_send_char(uart_data);
            break;

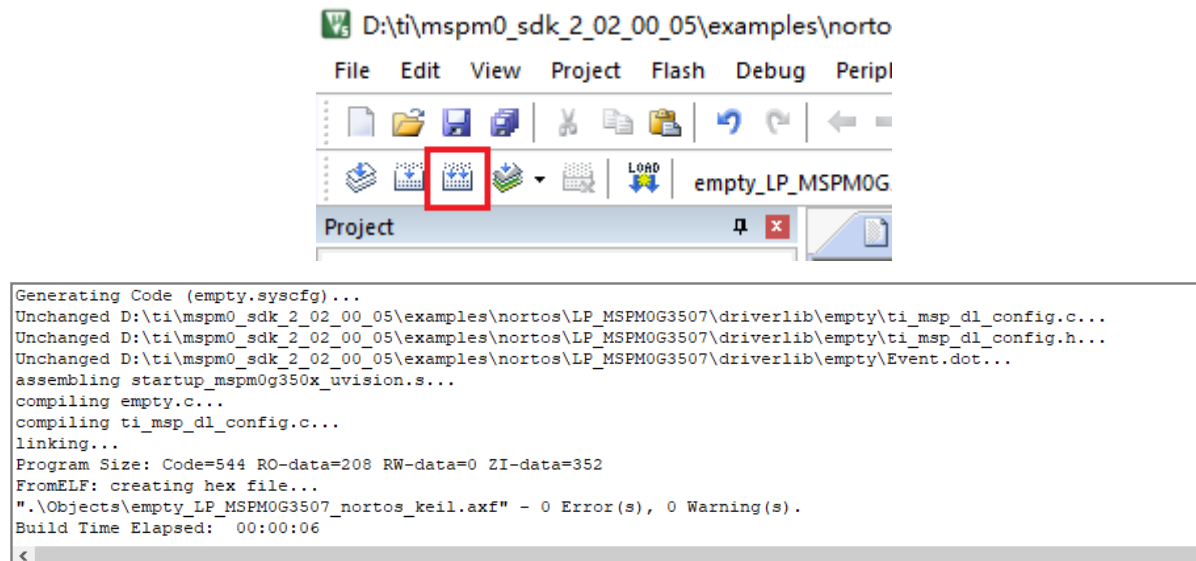
        default://其他的串口中断 Other serial port interrupts

```

```
        break;
    }
}
```

5. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



四、程序分析

- empty.c

```
int main(void)
{
    SYSCFG_DL_init();
    //清除串口中断标志 clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //使能串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);

    while (1)
    {
        //LED引脚输出高电平 LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(500);
        //LED引脚输出低电平 LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(500);
    }
}
```

串口中断必须手动开启。函数 `NVIC_EnableIRQ` 指定开启某一个中断。开启之前要先清除中断标志位，否则开启中断后将会自动进入中断。

```
//串口发送单个字符 Send a single character through the serial port
void uart0_send_char(char ch)
{
    //当串口0忙的时候等待，不忙的时候再发送传进来的字符
    // wait when serial port 0 is busy, and send the incoming characters when it
    is not busy
```

```

while( DL_UART_isBusy(UART_0_INST) == true );
//发送单个字符 Send a single character
DL_UART_Main_transmitData(UART_0_INST, ch);
}
//串口发送字符串 Send string via serial port
void uart0_send_string(char* str)
{
    //当前字符串地址不在结尾 并且 字符串首地址不为空
    // The current string address is not at the end and the string first address
    is not empty
    while(*str!=0&&str!=0)
    {
        //发送字符串首地址中的字符，并且在发送完成之后首地址自增
        // Send the characters in the first address of the string, and increment
        the first address after sending.
        uart0_send_char(*str++);
    }
}

```

定义两个函数，`uart0_send_char` 函数通过串口发送单个字符，`uart0_send_string` 函数通过串口发送字符串。

```

//串口的中断服务函数 Serial port interrupt service function
void UART_0_INST_IRQHandler(void)
{
    //如果产生了串口中断 If a serial port interrupt occurs
    switch( DL_UART_getPendingInterrupt(UART_0_INST) )
    {
        case DL_UART_IIDX_RX://如果是接收中断 If it is a receive interrupt
            //接发送过来的数据保存在变量中 The data sent is saved in the variable
            uart_data = DL_UART_Main_receiveData(UART_0_INST);
            //将保存的数据再发送出去 Send the saved data again
            uart0_send_char(uart_data);
            break;

        default://其他的串口中断 other serial port interrupts
            break;
    }
}

```

`UART_0_INST_IRQHandler` 函数实现串口接收数据并回传的功能。通过 `DL_UART_getPendingInterrupt()` 获取当前 UART0 的中断类型，如果接收到了数据触发的UART 中断，就将接收到的数据保存在变量中，之后再发出去。

五、实验现象

程序下载完成之后，可以通过串口调试助手发送数据给开发板，就能看到开发板返回的数据。

